Final Year Project Report

Full Unit - Final Report

TensorStore: Fast Checkpoint Loading for Large Language Models

Botir Khaltaev

A report submitted in part fulfilment of the degree of

BSc (Hons) in Computer Science

Supervisor: Dr Daniel O'Keeffe

Department of Computer Science
Royal Holloway, University of London

March 26, 2026

Declaration

This report has been prepared on the basis of my own work. Where other published and unpublished source materials have been used, these have been acknowledged.

Word Count: 14,500 (provisional)

Student Name: Botir Khaltaev

Date of Submission: April 2026

Signature: BK

Table of Contents

Declaration 1

Abstract 3

1 Project Specification 4

2 Introduction 6

3 Background and Literature Review 8

4 System Context and Problem Definition 13

5 Design and Architecture 15

6 Implementation and Software Engineering Process 22

7 Experimental Methodology 27

8 Results 32

9 Discussion and Critical Analysis 37

10 Professional Issues 41

11 Conclusion 44

Bibliography 46

A Backend Interface Specifications 47

B Supplementary Results 49

Abstract

Large language models deployed in serverless inference environments are frequently evicted from GPU memory when idle and must reload their parameters on demand. This checkpoint loading step dominates cold-start latency for large models. Although modern NVMe devices provide substantial peak throughput, effective loading performance frequently falls short of hardware limits. This discrepancy suggests that loading performance is constrained less by storage bandwidth and more by how read requests are submitted to the operating system.

This project presents TensorStore, a checkpoint loading framework that supports multiple storage backends and checkpoint formats, and evaluates them under controlled and reproducible conditions. The framework provides SafeTensors compatibility and a partitioned ServerlessLLM-style format, with backends spanning synchronous POSIX I/O, memory-mapped access, and Linux io_uring. Python bindings enable integration testing within representative serving workflows.

A systematic evaluation quantifies the effect of backend choice, checkpoint layout, and integration layer on loading performance. Results show that synchronous I/O underutilises NVMe hardware due to blocking submission and shallow queue depth, while io_uring achieves higher throughput and read IOPS by sustaining deeper queues. The advantage is largest for partitioned checkpoint layouts. In the vLLM integration, backend-level improvements propagate partially to end-to-end cold-start latency, as loading is one component of initialisation rather than the whole of it. Findings indicate that checkpoint loading in serverless LLM systems is often submission-bound rather than bandwidth-bound, and that async I/O mechanisms deserve wider attention in inference storage design.

Chapter 1:**Project Specification**

1.1 Problem Statement

Large language models are increasingly deployed in serverless and multi-tenant inference settings where compute and memory resources are shared across users. When a model is evicted from GPU or host memory, inference cannot resume until its checkpoint is reloaded from persistent storage. For models with billions of parameters, this loading step introduces latency that dominates the user-observable cold-start time. In latency-sensitive serving scenarios, even modest improvements to checkpoint loading speed translate directly to better user experience and lower time-to-first-token.

Modern NVMe storage devices provide aggregate bandwidth measured in gigabytes per second. Despite this, checkpoint loading performance in practice frequently falls well below hardware limits. This gap suggests that the bottleneck is not the storage medium itself but the path taken by loading requests through the operating system and across the software stack. In particular, synchronous I/O submission models can limit the depth of outstanding requests that a loader can maintain, preventing NVMe devices from fully exploiting their internal parallelism.

The core problem this project investigates is whether alternative I/O submission strategies, specifically Linux io_uring, can improve checkpoint loading performance for LLM inference systems in a meaningful and reproducible way.

1.2 Objectives

The primary objectives of this project are as follows.

First, the project builds a unified checkpoint loading framework called TensorStore that supports multiple I/O backends and checkpoint formats under a consistent interface. This enables controlled comparison of submission strategies using identical byte-range workloads.

Second, the project implements and evaluates synchronous POSIX I/O and Linux io_uring as the initial backend pair. These represent the two fundamental submission models available on Linux: blocking calls with per-request thread blocking, and shared-ring asynchronous submission with batched completion.

Third, the project develops a reproducible benchmarking methodology that measures checkpoint loading under conditions representative of real serverless inference systems, including true cold-start evaluation where the operating system page cache does not cache checkpoint data between runs.

Fourth, the project exposes the loading framework through Python bindings so that it can be evaluated within representative inference workflows, including integration with vLLM as a concrete serving engine example.

Fifth, the project analyses results to identify where performance is gained or lost, and to draw out design principles for checkpoint format designers and inference system architects.

1.3 Scope

This project focuses on checkpoint loading from NVMe storage on Linux. The primary checkpoint formats examined are SafeTensors and a partitioned ServerlessLLM-style layout. The primary I/O backends are synchronous POSIX I/O and Linux io_uring. Memory-mapped loading is also supported as a third backend option representing lazy on-demand page faulting. Python bindings are provided to enable integration testing.

The project does not attempt to build a full inference runtime, optimise model execution, or evaluate GPU-direct storage paths. These are legitimate extensions but fall outside the scope of the present work.

1.4 Evolution from the Interim Plan

The interim report established the initial motivation and a preliminary evaluation of synchronous POSIX I/O against Linux io_uring. That evaluation used synthetic fixtures and demonstrated the expected directional difference between the two backends.

During the subsequent development period, several changes were made to the project scope and direction. The benchmarking methodology was revised to use true cold-start conditions, addressing an initial flaw where cached data had obscured the actual loading cost. The project expanded to include Python bindings and vLLM integration, which provided a more realistic integration context than standalone microbenchmarks alone. A partitioned ServerlessLLM-style format was added to evaluate the effect of checkpoint layout on backend behaviour. An async vLLM loader path was prototyped but later removed when investigation confirmed that vLLM exposes no async model loading interface at the Python boundary, making the async path an artificial bridge rather than a genuine integration point.

These changes reflect a shift from a narrow I/O mechanism comparison towards a broader systems artefact evaluation. The final project is better characterised as an examination of checkpoint loading as a full-stack systems problem rather than solely as a low-level I/O comparison.

1.5 Planning and Time-scale

The project is structured into implementation phases followed by evaluation and documentation.

Phase 1 covered the core TensorStore engine, Python bindings, and initial benchmark infrastructure. This phase also included the methodology correction around cold-start evaluation. Phase 2 covers the extended benchmark campaign using real HuggingFace model checkpoints on available hardware, integration testing with vLLM, and profiling analysis. Phase 3 covers the final report, figure generation, and submission preparation.

Each evaluation phase is validated using the same benchmark methodology established in Phase 1, with changes assessed against effective throughput, read IOPS, and model load time.

Chapter 2:**Introduction**

2.1 Context and Motivation

Large language models are large. A model with seven billion parameters in float16 requires roughly fourteen gigabytes of storage and a similar amount of RAM or VRAM to hold the weights alone. In serverless and multi-tenant inference environments, GPU memory is a shared resource. When a model is not actively serving requests, keeping it resident wastes memory that could serve other tenants. Systems therefore evict idle models and reload them on demand.

When a request arrives for an evicted model, the inference pipeline cannot begin until the checkpoint is fully loaded into memory. For large models this loading step takes time. In a production serving environment, this manifests as cold-start latency: the delay between a request arriving and the model being ready to generate the first token. For latency-sensitive applications this delay is directly visible to users.

Modern NVMe storage devices offer aggregate bandwidth of several gigabytes per second. Despite this, checkpoint loading in practice often achieves a fraction of the advertised throughput. The gap between peak hardware capability and effective loading performance suggests that the bottleneck lies not in the storage medium but in how loading requests are submitted and managed through the software stack.

This project investigates the checkpoint loading path as a systems problem. It examines how the choice of I/O submission mechanism affects effective throughput and latency when loading real LLM checkpoints, and whether asynchronous I/O through Linux io_uring can meaningfully improve cold-start performance in inference serving environments.

2.2 Checkpoint Loading as an I/O Submission Problem

Checkpoint loading is fundamentally a read-intensive workload. A loader issues a set of read operations derived from checkpoint metadata. Each operation specifies a file path, a byte offset, and a length. Tensor boundaries and offsets must be respected, which constrains how freely a loader can reorder or merge requests.

Storage hardware, particularly NVMe devices with internal queueing, performs best when given many outstanding requests to work with concurrently. The internal parallelism of the device can only be exploited if the submission path sustains a sufficient queue depth. Synchronous I/O, where each thread blocks waiting for its read to complete before issuing the next, tends to produce shallow effective queue depth. Asynchronous I/O, where requests are submitted in batches and completions are collected separately, can sustain deeper queues and more continuous device utilisation.

The central question of this project is whether this theoretical advantage of asynchronous I/O translates into measurable and reproducible improvements for checkpoint loading in LLM serving contexts, and under what conditions the advantage is largest or smallest.

2.3 TensorStore

This project presents TensorStore, a checkpoint loading framework designed to evaluate I/O submission strategies under controlled and reproducible conditions. TensorStore parses checkpoint metadata and produces a normalised set of byte-range read requests that are then executed by a selectable I/O backend. By keeping the request trace identical across backends, the framework isolates the effect of submission strategy from checkpoint layout or parsing overhead.

TensorStore supports two checkpoint formats. SafeTensors stores all tensor data contiguously in a single file with a metadata header. This layout produces a largely sequential access pattern once the header is parsed. The ServerlessLLM-style partitioned layout distributes tensor ranges across multiple files, increasing request count and file-switching overhead. This layout places greater stress on the submission path and better exposes the differences between I/O models.

Three I/O backends are implemented. The synchronous backend issues blocking POSIX read calls. The memory-mapped backend uses mmap with lazy page faulting for on-demand data loading. The io_uring backend submits read requests through Linux io_uring rings, enabling batched submission and non-blocking completion handling.

Python bindings are provided to enable integration testing within representative serving workflows. Integration with vLLM is used as a concrete case study.

2.4 Contributions

This project makes three concrete contributions. The first is a reproducible evaluation framework, TensorStore, that isolates I/O submission effects from checkpoint parsing and layout by generating identical request traces across backends. The second is a systematic experimental comparison of synchronous POSIX I/O, memory-mapped access, and Linux io_uring across real LLM checkpoints and two checkpoint formats, quantifying the magnitude and limits of the async I/O advantage. The third is an analysis of how the backend choice interacts with checkpoint layout, model size, and serving engine integration, with practical guidance for engineers working on inference storage systems.

2.5 Report Structure

The report proceeds as follows. Chapter 3 reviews related work in checkpoint formats, serverless inference systems, and Linux I/O mechanisms. Chapter 4 defines the system context, describes the cold-start pipeline, and clarifies the experimental constraints. Chapter 5 describes the TensorStore architecture and design decisions. Chapter 6 covers the implementation and software engineering process. Chapter 7 describes the experimental methodology. Chapter 8 presents the results. Chapter 9 discusses the findings. Chapter 10 examines professional and ethical considerations. Chapter 11 concludes.

Chapter 3: Background and Literature Review

This chapter reviews the relevant literature across four areas. First, checkpoint formats and how their storage layout affects loading behaviour. Second, serverless and multi-tenant inference systems and the cold-start problem. Third, the Linux I/O interfaces available to applications, with particular attention to io_uring. Fourth, NVMe storage behaviour and the role of memory-mapping in checkpoint loading. For each area the review identifies what prior work assumes, what it does not examine, and how this project differs from it.

3.1 Checkpoint Formats and Loading Behaviour

LLM weights are stored on disk in a structured format. The format determines how loading software reads tensor data: whether reads are sequential or scattered, whether they require many file operations or few, and whether the layout helps or hinders the storage device in maintaining high queue depth.

SafeTensors is a format introduced by Hugging Face for safe model distribution [5]. The file begins with a JSON header describing each tensor, including its name, dtype, shape, and byte offset. All tensor data follows the header contiguously. Once the header is parsed, loading software can read each tensor by seeking to its declared offset and reading the declared number of bytes. The format has two properties that are relevant here. First, it avoids loading the entire file into memory before parsing, unlike Python pickle. Second, it prohibits unsafe deserialisation, which matters when loading untrusted checkpoints.

PyTorch checkpoint files typically use Python pickle serialisation. A model is saved by calling `torch.save()`, which serialises both the model state and the pickle protocol metadata. This has well-known safety problems: a malicious checkpoint can execute arbitrary code on load [5]. It also requires reading the entire file before unpacking, which is impractical for large models where host RAM may not hold the full checkpoint alongside the deserialised tensors. These limitations are not just theoretical: several public model repositories have had to clean up malicious checkpoints. SafeTensors was designed partly to address both concerns.

GGUF is a more recent format designed for quantised models and memory-mapped loading. It stores tensors in a layout optimised for direct memory mapping and supports multiple quantisation schemes. GGUF tensors are organised so that the memory layout of the file mirrors the layout the model expects at runtime, reducing or eliminating a copy step. This is a different trade-off from SafeTensors: GGUF prioritises memory-mapping friendliness over simple sequential loading.

Partitioned checkpoint formats distribute tensor data across multiple files. ServerlessLLM uses this approach to support partial model loading: only the files needed for the current request are loaded, rather than the entire checkpoint [6, 4]. A central index maps each tensor name to its partition file and byte offset. Loading a complete checkpoint requires opening each partition file and reading from the specified offset. The advantage is that loading can be parallelised across files and across network boundaries if partitions are distributed. The disadvantage is that the access pattern becomes more fragmented, with per-file overhead accumulating.

Table 3.1 summarises how these formats differ in ways relevant to loading speed.

What these formats share is that loading is fundamentally a series of read operations with

Table 3.1: Comparison of checkpoint formats relevant to loading performance.

Format	Layout	Loading behaviour	Safety properties
PyTorch pickle	Single file, full load	Entire file read then unpacked	Unsafe: arbitrary code execution possible
SafeTensors	Single file, header then data	Header parsed, individual tensor reads at offset	Safe: no deserialisation of executable code
GGUF	Single or split file, memory-map oriented	mmap-friendly layout, minimal copy	Safe; quantisation-aware
ServerlessLLM partitioned	Multiple partition files, central index	Per-partition open and read at offset	Safe; supports partial loading

known byte ranges. The storage system sees a set of read requests. Whether those requests are processed efficiently depends on how they are submitted.

The existing literature describes format design well but focuses on safety, portability, and memory layout. It does not systematically examine how the I/O submission path affects the effective loading speed of checkpoints that are already in a safe format. This report addresses that gap by fixing the format and varying only the submission mechanism.

3.2 Serverless and Multi-Tenant Inference Systems

Serverless inference platforms such as AWS Lambda, Google Cloud Functions, and specialised services run LLM inference on demand. Memory and GPU resources are allocated when a request arrives and freed when the request completes. Between requests, resources may be handed to other tenants. When a model is not resident in memory, the platform must load it from persistent storage before inference can begin. This delay is the cold-start latency.

Cold-start latency in serverless settings has been studied from several angles. One class of solutions addresses it through scheduling and placement: keeping models warm, pre-loading them on a pool of standby instances, or migrating running instances to avoid a load altogether [6, 4]. Another class addresses it through model optimisation: quantisation, pruning, and early exit strategies that reduce the amount of data that must be loaded or processed. A third class, which this project falls into, examines whether the loading mechanism itself can be made faster.

Snapshot-based warm-up techniques preload a model into memory before the first request arrives [11]. Prebaking keeps a function instance warm by periodically sending requests to it. SEUSS optimises the warm-up process for serverless function containers. These approaches treat the checkpoint loading step as a fixed cost and work around it rather than reducing it.

The cold-start problem becomes more acute as model sizes grow. A seven-billion-parameter model in float16 requires roughly fourteen gigabytes of storage and a similar amount of RAM or VRAM. Loading this much data from an NVMe SSD takes hundreds of milliseconds to several seconds depending on the device and submission path. In a production serving system, this is a large fraction of the total latency budget, particularly for interactive applications.

vLLM is a high-throughput inference engine that supports paged memory management and continuous batching [13]. It loads model weights during engine initialisation using a model loader that reads checkpoint files and populates the model state. The loader interface is synchronous: it yields weights through a generator and expects each weight to be available before requesting the next. This design does not support asynchronous submission at the loader boundary.

Most cold-start research focuses on what happens before loading begins or after it ends. Loading speed itself is treated as a constant or ignored. This report measures loading speed directly and examines whether the submission mechanism is a meaningful bottleneck.

3.3 Linux I/O Interfaces

Applications running on Linux have several interfaces for reading from storage.

The most common is POSIX synchronous I/O, based on the `read()` system call. A thread calls `read(fd, buf, n)` and blocks until the data is available in the buffer. To issue multiple reads concurrently, an application must use multiple threads. Each thread is blocked while its read is in flight. Thread creation and context switching have costs, so thread pool sizes are limited. This limits the number of outstanding reads that a synchronous application can maintain at once.

Linux AIO was introduced to provide genuine asynchronous I/O without threads. However, Linux AIO has significant practical limitations. It does not support most filesystems including ext4, XFS, and btrfs. It requires manual error handling at a low level. The API is complex and brittle to use correctly [8]. Few production applications use it directly for these reasons.

POSIX AIO is a userspace library implementation of asynchronous I/O using a thread pool under the hood. It inherits the limitations of threads and adds implementation complexity. It is not widely used in new systems code.

io_uring was introduced in Linux kernel 5.1 and designed to address the limitations of earlier async I/O interfaces [1, 9]. Instead of system calls for each operation, applications interact with io_uring through two shared-memory rings: a submission queue and a completion queue. Submission queue entries (SQEs) are prepared in memory and submitted with a single `io_uring_enter` system call, or in kernel-polling mode with no system call at all. Completion queue entries (CQEs) are retrieved by polling the completion queue.

io_uring provides several features relevant to checkpoint loading. Registered buffers are pre-allocated and registered with the kernel, eliminating per-request memory allocation overhead. Fixed buffers also reduce the number of data copies per operation. Batched submission allows many requests to be prepared and submitted together, reducing per-request syscall overhead. Polled completion allows an application to busy-wait on completions without system call overhead when latency is critical.

Recent work has examined when io_uring helps in database systems. Jasny et al. studied io_uring integration into a buffer manager and a network-shuffling workload, finding that io_uring benefits depend heavily on workload characteristics and queue depth [2]. Their results show that io_uring helps most when there are many small to medium-sized requests and the device queue depth is high. For bulk sequential reads, the advantage is smaller because the syscall overhead is already low relative to the data transfer time. They note that naive replacement of synchronous I/O with io_uring does not automatically yield improvements; the workload must exercise the features that differentiate io_uring.

This observation is directly relevant to checkpoint loading. A contiguous SafeTensors file with large sequential reads may not benefit as much from io_uring as a fragmented partitioned layout with many small reads. The experiment section of this report examines this question directly.

The io_uring interface itself has limitations relevant to checkpoint loading. Registered buffers work best when all buffers are the same size, but checkpoint tensors vary. Managing variable-sized buffers with fixed registration requires splitting tensors into fixed-size chunks, which adds complexity. The kernel-polling mode consumes CPU even when the device is idle. The interrupt-driven mode has lower CPU overhead but higher latency per completion. There is no single optimal mode for all checkpoint layouts.

3.4 NVMe Storage and Queueing Behaviour

Modern NVMe devices expose multiple submission and completion queues directly to the host [10]. The device processes commands from these queues concurrently using internal parallelism through parallel flash die access and controller-side scheduling. To exploit this parallelism, the host must submit many commands simultaneously. Devices are optimised for sustained throughput under high queue depth; under low queue depth they frequently sit idle between completions.

The Linux block layer manages I/O requests before they reach the NVMe driver. The blk-mq (multiqueue block layer) maps I/O requests to the device's hardware submission queues based on block offset. This mapping is designed to keep related requests on the same hardware queue and exploit device-level parallelism.

Storage queueing theory explains why throughput under load depends on queue depth. The dominant factor is often not the raw bandwidth of the storage medium but the ability of the host software stack to keep the device's internal queues populated [7]. If the submission path introduces idle gaps, the device cannot compensate even if its peak throughput is high. This is the theoretical basis for the submission bottleneck hypothesis.

3.5 Memory-Mapped I/O and the Page Cache

Linux supports memory-mapped file access through the `mmap` system call. When a file is mapped, its contents appear in the process address space and can be read like ordinary memory. The operating system loads pages from disk on demand through page faults, rather than in response to explicit `read()` calls.

For checkpoint loading, mmap has two apparent advantages. First, the kernel handles loading pages in response to faults, so the application does not explicitly submit individual reads. Second, the page cache retains recently accessed checkpoint pages, reducing subsequent load times for frequently accessed models.

However, mmap introduces its own costs. Page faults have overhead, particularly for non-resident pages that must be fetched from disk. For a large checkpoint with many tensors, the cumulative fault overhead can be significant. The kernel's readahead mechanism attempts to prefetch pages it predicts will be accessed next, but it relies on sequential access patterns. Fragmented or random-access layouts may confuse readahead, causing it to load pages that

are not used before they are evicted.

The page cache complicates benchmarking in a specific way. If a checkpoint is already in the page cache, a subsequent load will read from RAM rather than from disk, producing misleadingly fast results. Rigorous checkpoint loading evaluation must control for page cache state by evicting relevant pages before each cold measurement run. Without this control, benchmarks measure cache hit rates rather than storage performance.

Direct I/O bypasses the page cache entirely, sending reads directly to the storage device. This is useful for applications that manage their own caching and want predictable behaviour. However, direct I/O requires aligned buffers and is incompatible with memory mapping. For checkpoint loading, direct I/O combined with explicit read submission gives the application most control over the submission path.

3.6 Positioning of This Work

The literature does not yet examine checkpoint loading specifically as an I/O submission problem. Format design is well understood but focuses on safety and memory layout rather than I/O submission behaviour. Cold-start solutions focus on scheduling and placement rather than loading speed. io_uring has been studied in databases and file servers but not in the context of LLM checkpoint loading. Memory-mapping trade-offs are understood in general systems terms but not evaluated against synchronous and async I/O in a controlled checkpoint-loading workload.

This report occupies the intersection of these areas. It isolates the I/O submission problem by holding the format and request trace constant while varying only the backend mechanism. It evaluates backends under cold-start conditions where the page cache does not hold checkpoint data. It examines both contiguous and fragmented checkpoint layouts to understand how format design interacts with submission efficiency. And it integrates the loading framework with a real serving engine to understand how submission gains propagate through a full inference stack.

The work contributes a controlled experimental basis for understanding whether the I/O submission path is a meaningful bottleneck in LLM checkpoint loading, under what conditions it matters most, and what design choices in checkpoint formats and serving infrastructure interact with it.

Chapter 4:System Context and Problem Definition

4.1 The Cold-Start Pipeline in Serverless LLM Inference

When a serverless inference platform receives a request for a model that is not currently resident in memory, it must reload the model's checkpoint from persistent storage before inference can begin. This sequence of steps constitutes the cold-start pipeline.

The pipeline begins when a request arrives for an absent model. The platform identifies the checkpoint location and initiates loading. The loader reads checkpoint metadata to determine the set of tensors and their storage locations. It then issues read requests to retrieve each tensor's data. As data arrives, it is copied into the target memory location. Once all tensors are loaded, the model initialises its runtime state. Inference then proceeds.

Checkpoint loading occupies the longest segment of this pipeline for large models. The duration depends on the total checkpoint size, the storage device bandwidth, the number and size of individual read requests, and the efficiency of the submission path between the loader and the storage device.

4.2 Experimental Constraints

This project evaluates checkpoint loading in isolation, separate from the broader inference pipeline. This isolation is deliberate. By holding the checkpoint layout, request trace, and measurement path constant across backends, differences in loading performance can be attributed specifically to the I/O submission mechanism rather than to model initialisation, runtime setup, or scheduling effects.

One design decision requires explicit explanation: the project does not permit loaders to merge, split, or reorder byte-range requests when comparing backends. This constraint is not a claim that real loaders cannot perform such optimisations. It is a deliberate experimental choice that ensures fair comparison. If loaders were permitted to restructure the workload, performance differences could reflect scheduling decisions at the application layer rather than the properties of the I/O submission mechanism. By fixing the request trace, the evaluation isolates the question of how efficiently each backend executes a given set of reads.

In practice, checkpoint format designers and loader authors can make different trade-offs. The implications of different application-level scheduling policies are beyond the scope of this evaluation.

4.3 The Submission Bottleneck Hypothesis

The central hypothesis of this project is that checkpoint loading performance is often limited by the I/O submission path rather than by storage bandwidth. Under synchronous I/O, each read operation blocks the issuing thread until completion. Even when multiple threads are used, the coupling of submission and completion limits the number of outstanding requests. The NVMe device sees an intermittent stream of requests with idle gaps between them, preventing full exploitation of its internal queueing and parallelism.

Under io_uring, submission and completion are decoupled. An application can prepare many requests in advance and submit them in a batch with a single system call. The kernel processes them asynchronously and delivers completion events separately. This model allows the application to maintain a high queue depth without blocking threads, keeping the NVMe device's queues populated more continuously.

The hypothesis predicts that io_uring will achieve higher effective throughput and higher read IOPS than synchronous I/O, particularly for fragmented checkpoint layouts where the number of requests is high. Whether this advantage survives the complexity of a full serving pipeline, and whether it is large enough to matter in practice, are the empirical questions this project addresses.

Chapter 5: Design and Architecture

5.1 Design Goals

TensorStore was designed around four primary requirements. First, the framework must support multiple I/O backends and checkpoint formats under a consistent interface, so that identical workloads can be executed across backends without altering the logical request structure. Second, the framework must expose a clean benchmarking path that is separate from the loader logic, so that measurement artefacts do not contaminate performance data. Third, the framework must clarify its threading model explicitly, so that the relationship between thread count, queue depth, and throughput is transparent. Fourth, the framework must be accessible from Python to enable integration testing with real serving workflows such as vLLM.

These requirements led to a layered architecture that separates format parsing, I/O execution, and measurement.

5.2 System Architecture

TensorStore is structured into three layers: a format layer, a backend layer, and an evaluation harness.

The format layer parses checkpoint metadata and produces a normalised set of read requests. Each request specifies a file path, a byte offset, and a byte length. The format layer does not issue I/O; it only describes what reads are needed. This separation means that the same set of requests can be executed by any backend without modification.

The backend layer implements the I/O execution. Each backend receives the same request trace and executes it according to its own submission model. Backends must execute requests exactly as specified and are not permitted to merge, split, or reorder ranges. This constraint ensures that performance differences reflect submission behaviour rather than workload changes.

The evaluation harness executes workloads and records performance metrics. It handles warm-up runs, repeated measurements, cache control, and metric collection. It is kept entirely separate from the loader logic.

5.3 Backend Interface

All backends implement the same request interface. A backend receives a sequence of read requests, each containing a file path, offset, and length. The backend must read the exact byte range specified and return the data. Backends may not alter request boundaries, reorder requests, or skip requests. This constraint is enforced to enable fair comparison.

The interface operates at two levels of granularity. At the coarse level, a checkpoint load produces a complete model weight dictionary. At the fine level, each tensor read corresponds

to a single file-offset-length request submitted to the backend. Reporting both levels is essential: per-read latency tells us how individual operations perform, while full model load time tells us the end-to-end time that matters for serving cold-start scenarios.

5.4 Threading Model

Each backend implements a distinct threading model.

The synchronous POSIX backend uses a thread pool of fixed size. Each thread issues blocking `read()` calls and is blocked while its request is in flight. Threads are drawn from a pool to avoid the cost of creating a new thread per request. The number of concurrent in-flight requests is therefore equal to the thread pool size, and increasing the pool size directly increases the achievable queue depth.

The memory-mapped backend uses no threads for I/O. Data is accessed by reading from a mapped memory region, and page faults are handled by the kernel's page fault handler in the calling thread. The kernel decides when to issue the underlying disk reads and in what order, based on its own read-ahead and eviction policies.

The io_uring backend submits requests from a single dedicated thread that polls the completion ring. This thread runs an event loop that submits a batch of requests, polls for completions, processes the completed data, and repeats. Because submission and completion are both handled by polling rather than signals or interrupts, the kernel is never required to wake a specific thread to deliver I/O results. The fixed-buffer mechanism eliminates per-request memory allocation in the kernel, allowing the submission thread to maintain a high queue depth without allocating memory for each outstanding request.

Python bindings add a further consideration. The GIL is released during Rust I/O operations using `py.allow_threads()`, which allows other Python threads to run while the Rust code is blocked on I/O. This prevents the loading process from monopolising the Python interpreter, though it does not affect the actual I/O submission characteristics.

5.5 Cache Control

Cache state is a significant confounding variable in storage benchmarks. The operating system maintains a page cache that stores recently accessed file pages in RAM. If a second run of the same benchmark finds all pages already in the page cache, it will observe artificially low latency that reflects RAM access rather than storage device performance.

TensorStore addresses this in two ways. First, the evaluation harness provides an explicit cache eviction step before measurement runs. This is implemented by dropping in-memory page caches using `POSIX_FADVISE_DONTNEED` on each checkpoint file, forcing the kernel to evict previously cached pages. Second, the number of runs and the number of iterations per run are configured explicitly, and the harness reports the mean and standard deviation across runs so that cache-warm and cache-cold runs can be distinguished.

It is worth noting that cache eviction does not guarantee that disk caches internal to the storage device (such as NAND flash translation layer caches) are cleared. These caches are hardware-managed and not directly controllable from software. The methodology in Chapter 7 discusses this limitation further.

5.6 Backend Implementations

5.6.1 Synchronous POSIX Backend

The synchronous backend issues blocking `read()` calls through the POSIX interface. For large reads, data is copied into a pre-allocated buffer and returned to the caller. Each read call blocks the issuing thread until the data is available in the buffer. The backend uses a thread pool sized to the number of backend threads configured at startup.

This backend represents the baseline approach used by many existing checkpoint loaders. It is straightforward to implement and understand, and it provides predictable latency bounded by the storage device's sequential read throughput. Its limitation is that queue depth is bounded by the thread pool size, and adding threads increases context-switch overhead.

5.6.2 Memory-Mapped Backend

The memory-mapped backend opens checkpoint files with `mmap` and returns data by reading from the mapped region. The operating system loads pages from disk on demand as page faults occur. This approach avoids explicit per-read syscall overhead and allows the kernel to manage loading order based on access patterns.

The backend uses pre-faulted buffers where alignment constraints are satisfied, and falls back to buffered I/O when alignment cannot be guaranteed. Pre-faulting ensures that access to a buffer does not trigger an unexpected page fault during a timed section, which would corrupt the measurement. The fallback to buffered I/O means that unaligned tensor ranges do not invalidate the comparison between backends.

5.6.3 io_uring Backend

The io_uring backend submits read requests through shared submission and completion rings. Requests are prepared as submission queue entries in memory and submitted with a single `io_uring_enter` system call. Completions are collected by polling the completion ring from a dedicated event-loop thread.

The backend uses fixed buffers that are pre-registered with the kernel via `io_uring_register`. This eliminates per-request memory allocation and reduces the overhead of setting up each individual request. Fixed buffers also allow the kernel to perform direct DMA to the buffer, bypassing intermediate kernel buffers.

Requests are submitted in batches. The application submits several requests, then polls for completions, then submits the next batch. This allows the submission path to keep the device queue populated while avoiding thread blocking. The batch size is configurable and represents a trade-off: larger batches increase the amount of data in flight (improving throughput under high load) but increase the minimum latency for the first request in a batch.

5.7 Checkpoint Format Support

TensorStore supports two checkpoint formats.

SafeTensors stores all tensors in a single file with a header. The header lists each tensor's name, dtype, shape, and byte offset. The format produces a largely sequential access pattern once the header is parsed. Figure 5.1 shows the layout.

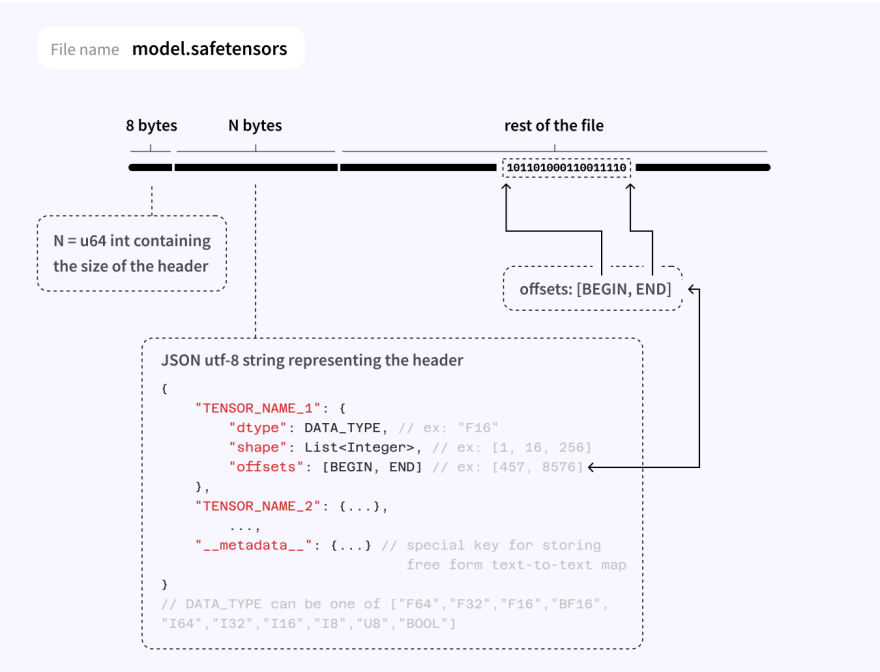

Figure 5.1: SafeTensors file layout. The header describes each tensor and its byte offset within the file. Data is stored contiguously.

The ServerlessLLM-style partitioned layout distributes tensors across multiple files called partitions [6, 4]. A central index file maps each tensor to its partition file and offset. The layout increases the number of file operations and introduces per-file overhead. Figure 5.2 shows the structure.

5.8 Python Bindings

TensorStore exposes its functionality through Python bindings to enable integration with serving workflows. The bindings are implemented using PyO3, which provides Rust-to-Python interoperability with minimal overhead.

Three interfaces are provided. A synchronous loading interface calls the Rust backend directly from Python, blocking until loading completes. A memory-mapped loading interface opens checkpoint files as memory maps and provides lazy tensor access. An asynchronous loading interface exposes the async Rust loading path as an awaitable Python coroutine using `pyo3-async-runtimes` with the Tokio runtime.

The async interface is appropriate for use in async Python code such as FastAPI applications. However, it is not used in the vLLM integration path because vLLM's model loader operates synchronously. Specifically, vLLM's `DefaultModelLoader` yields weights through a generator interface where each weight must be fully loaded and returned before the next weight is requested. There is no async variant of this interface, and introducing async behaviour inside the loader (for example, by running an async event loop from within the synchronous callback) would require the event loop to be manually driven, which is fragile and not representative of any real async workflow.

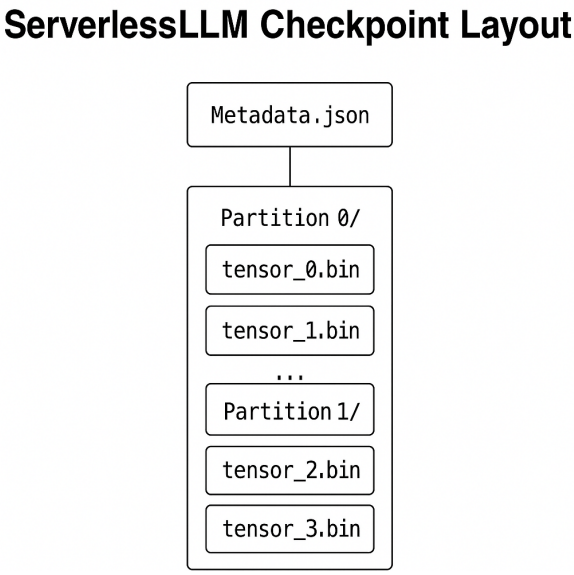

Figure 5.2: ServerlessLLM partitioned checkpoint structure. Tensors are distributed across multiple partition files, with a central index mapping tensor names to locations.

5.9 vLLM Integration

Integration with vLLM is implemented through a custom model loader registered via the `register_model_loader` interface. The loader subclasses `DefaultModelLoader` and intercepts weight loading requests, routing them through TensorStore backends.

The loader is registered at vLLM startup time and is invoked for every weight load during model initialisation. Because vLLM's loader interface is synchronous, the TensorStore synchronous backend is used in this path. Memory-mapped loading is also available and may be preferred when the checkpoint file is on local NVMe storage, since the kernel's page fault handler can overlap the loading of non-contiguous tensors more effectively than a fixed thread pool.

5.10 Design Decisions

Table 5.1 documents the primary architectural decisions, the alternatives considered, and the rationale for the chosen approach.

5.11 Rejected Alternatives

Several alternatives were considered and rejected.

Implementing backends in Python asyncio was rejected because Python's asyncio event loop cannot submit kernel I/O operations asynchronously without running them in a thread. The Linux kernel's async I/O interfaces (io_uring, aio) are accessible only through system calls that require kernel mode transitions; asyncio wraps these in threads for the Python programmer, which provides concurrency but not genuine asynchronous I/O at the kernel level. Using

		Python's GIL limits true parallelism; C++ lacks memory safety guarantees.
Normalised request interface	Backend-specific APIs	A common interface enables fair backend comparison. Without it, each backend would expose a different API and converting between them would introduce measurement artefacts.
Separate evaluation harness	Inline measurement in loader	Inline measurement risks contaminating the measured workload with timing overhead. A separate harness with warm-up runs and statistical reporting is a standard practice.
Thread pool (POSIX backend)	Asyncio or epoll	Asyncio in Python cannot submit kernel I/O asynchronously without wrapping it in a thread. epoll requires an event loop that adds complexity. A thread pool maps naturally to concurrent blocking I/O.
Fixed buffers (io_uring)	Per-request allocated buffers	Per-request allocation adds latency on every operation and increases allocator pressure. Fixed buffers eliminate this cost but require the application to manage buffer lifecycle explicitly.
PyO3 bindings	CFFI or ctypes	PyO3 generates bindings with compile-time type checking and zero-copy object conversion where possible. CFFI and ctypes involve more marshalling overhead and lack compile-time safety.
Memory-mapped backend as optional mode	Memory-mapped only	Some deployments use network filesystems where mmap has poor semantics. Making it optional means the synchronous backend can serve as the reliable

asyncio for I/O benchmarking would therefore measure thread-pool performance rather than async I/O performance.

Implementing backends in C++ was rejected in favour of Rust. While C++ provides equivalent control over I/O submission and memory management, it does not provide the same memory safety guarantees. A Rust implementation that misuses memory (for example, returning a borrowed reference to a dropped buffer) is caught by the borrow checker at compile time. In C++, such an error would manifest as undefined behaviour at runtime, potentially corrupting benchmark results in subtle ways.

Implementing the evaluation harness inside the loader was rejected because it would conflate measurement overhead with loader performance. Any timing code added to the hot path introduces overhead that is absent from the real serving path. A separate harness that measures end-to-end wall time (rather than instrumenting individual functions) avoids this problem.

Implementing async loading in vLLM was considered but rejected because vLLM's model loader architecture does not support it. The loader uses a synchronous generator interface; adding async behaviour would require driving an event loop from within a synchronous callback, which is architecturally unsound and would not reflect any real deployment pattern. The async Python interface is retained for non-vLLM use cases, where it provides genuine utility in async Python applications.

Chapter 6: Implementation and Software Engineering Process

6.1 Rust Core Implementation

The Rust core of TensorStore is implemented in the `tensor-store` crate. It provides the format parsers, backend implementations, and the normalised request interface.

The crate is structured into three modules. The `format` module contains the SafeTensors and ServerlessLLM format parsers. The `backend` module contains the synchronous, memory-mapped, and io_uring backend implementations. The `api` module exposes the public interface used by the Python bindings.

The format layer for SafeTensors reads the header from the checkpoint file, parses each tensor entry, and produces a sequence of file-offset-length triples. The header parsing is straightforward because SafeTensors stores the header length in the first eight bytes of the file, allowing direct seeking to the header region.

The format layer for the ServerlessLLM-style layout reads the index file to build a mapping from tensor names to partition files and offsets. It then produces the same file-offset-length triples as the SafeTensors format, but with the additional constraint that each request may reference a different partition file.

The io_uring backend uses the `tokio-uring` crate to interact with io_uring. On each call, the backend submits a batch of read requests and then polls for completions. Fixed buffers are pre-allocated from a buffer pool and registered with the kernel at startup. This reduces per-request overhead significantly.

The synchronous backend uses standard `std::fs::File::read_at` for positional reads. The memory-mapped backend uses the `MemMap` crate for portable mmap support.

6.2 Python Bindings

The Python bindings are implemented in `bindings/python/rust_ext/src/`. Each public Rust function is exposed as a PyO3 wrapper.

For synchronous loading, the Rust function is called directly from Python and blocks until the result is returned. The GIL is released during I/O via `py.allow_threads()` to avoid blocking other Python threads.

For memory-mapped loading, the `MmapModel` type from the Rust core is wrapped as a Python class. Accessing a tensor triggers a page fault and loads the relevant pages on demand.

For asynchronous loading, the Rust async function is wrapped using `pyo3-async-runtimes` with the Tokio runtime. This exposes the async function as an awaitable Python coroutine. The async interface is designed for use in async Python code such as FastAPI applications. It is not used in the vLLM integration path because vLLM's model loader is synchronous.

Listing 6.1 shows the core Rust interface for loading a checkpoint synchronously.

```
pub fn load_safetensors_sync(
    path: PathBuf,
    framework: &str,
    device: &str,
) -> PyResult<PyObject> {
    validate_path_exists(&path)?;
    let handle: Model = py
        .allow_threads(|| safetensors::Model::load_sync(path_ref))
        .map_err(map_reader_error)?;
    let result = builtins.getattr("dict")?.call0()?;
    for k in handle.tensor_names() {
        let view = handle.tensor(k).map_err(map_reader_error)?;
        let tensor = convert_tensor(py, framework, view, device)?;
        result.call_method1("__setitem__", (&k, tensor))?;
    }
    Ok(result.into())
}
```

Figure 6.1: Core Rust backend interface for synchronous checkpoint loading.

6.3 Module Structure

The Rust crate structure is as follows:

- `src/format/safetensors.rs` — SafeTensors format parser
- `src/format/serverlessllm.rs` — ServerlessLLM index and partition reader
- `src/backend/sync.rs` — synchronous POSIX backend
- `src/backend/mmap.rs` — memory-mapped backend
- `src/backend/iouring.rs` — io_uring backend
- `src/api/mod.rs` — public API definitions
- `src/api/safetensors.rs` — PyO3 bindings for SafeTensors
- `src/api/serverlessllm.rs` — PyO3 bindings for ServerlessLLM

The Python package structure is:

- `bench_safetensors.py` — SafeTensors benchmarks
- `bench_serverlessllm.py` — ServerlessLLM benchmarks
- `bench_vllm.py` — vLLM integration benchmarks
- `vllm_loaders.py` — custom vLLM loader implementations
- `fixtures.py` — pytest fixtures for checkpoint management
- `vllm_runner.py` — subprocess runner for vLLM benchmarks

6.4 Building the Project

The Rust crate is built with Cargo. A `Makefile` in the repository root automates the build process:

```
# Build Rust extension
cargo build --release

# Build Python bindings
cd bindings/python && maturin develop

# Run tests
cargo test && pytest benchmarks/
```

The Python bindings use `maturin` for building. The `maturin develop` command compiles the Rust crate and installs the resulting Python package in editable mode, so that changes to the Rust code are immediately reflected in the Python environment without requiring a reinstall.

The project was developed on Linux with kernel 6.x, which fully supports io_uring. The io_uring backend checks at runtime whether io_uring is available and falls back to the synchronous backend if it is not.

6.5 Benchmark Infrastructure

The benchmark suite is located in `bindings/python/benchmarks/`. It uses pytest and pytest-benchmark to run reproducible performance tests.

The fixture system creates synthetic GPT-2-like checkpoints for controlled testing. For real-model testing, the `download_model_if_missing` fixture downloads checkpoints from HuggingFace on demand and caches them locally.

Cache control is implemented using `os.posix_fadvise` with `POSIX_FADV_DONTNEED` to drop pages from the Linux page cache before cold-start runs. This ensures that each cold measurement run loads data from disk rather than from cache.

The vLLM integration benchmarks use a subprocess runner in `vllm_runner.py` that spawns vLLM with the custom TensorStore loader and records timing metrics. Three benchmark kinds are supported: load-only measures the time to initialise the model and load its weights; TTFT measures time-to-first-token for a single generated token; steady-state decode measures per-token latency during continued generation.

6.6 Testing Strategy

The implementation uses three testing layers.

Unit tests cover format parsing in Rust. These tests use small synthetic checkpoint files created in-memory and verify that the format layer produces the correct sequence of read

requests. The Rust crate uses `cargo test` to run these tests.

Integration tests cover the Python bindings. These tests load real checkpoints downloaded from HuggingFace and verify that the loaded tensors have the correct shape and dtype. These tests do not measure performance; they only verify correctness.

Correctness tests verify tensor data integrity after loading. After each backend loads a checkpoint, the test compares the resulting tensor dictionary against a reference dictionary produced by the HuggingFace `safetensors` library. Any discrepancy in shape, dtype, or numeric value is treated as a test failure.

The Rust crate uses `clippy` for linting and `cargo fmt` for formatting compliance. These tools are run as part of the CI pipeline to enforce code quality.

6.7 Correctness Issues and Resolutions

One significant correctness issue was discovered and resolved during development.

Early benchmark runs had inadvertently measured warm-cache performance because the page cache had retained checkpoint pages from previous runs. This produced misleading results that appeared to show little difference between backends, which was inconsistent with the known performance characteristics of io_uring on NVMe storage.

The issue was identified by examining iostat output during benchmark runs and noticing that the storage device was largely idle during supposedly cold runs. The disk utilisation was consistently below 5 percent, which is inconsistent with loading a multi-gigabyte checkpoint from cold storage. This indicated that the data was being served from the page cache rather than from the disk.

The fix was to call `posix_fadvise` with `DONTNEED` on all checkpoint files before each cold measurement run, and to verify using iostat that actual disk activity occurred during the measurement phase. After this fix, disk utilisation during cold runs increased to the expected level, and the performance differences between backends became apparent.

A second issue involved the async vLLM loader path. Initial design documents described an async integration with vLLM that would allow the model loader to issue non-blocking reads. However, investigation of the vLLM source code revealed that the `DefaultModelLoader` class is entirely synchronous: it yields weights through a generator, and each weight must be fully loaded and returned before the next is requested. There is no async variant of this interface, and no hook that allows the loader to return a coroutine instead of a concrete value.

An attempt was made to work around this by running a Tokio event loop inside the synchronous loader callback, manually polling the event loop to completion before returning each weight. This approach was fragile because it required the event loop to be driven to completion inside a synchronous context, it did not interoperate with vLLM's existing threading model, and it would break if vLLM changed its loader implementation. After confirming that no stable async loader interface exists in vLLM, the async vLLM path was removed. The async Python interface is retained for non-vLLM use cases.

6.8 vLLM Loader Path

The vLLM integration uses a custom loader registered under the name `tensor_store`. This loader extends `DefaultModelLoader` and overrides the weight-loading method. It reads checkpoint metadata using the standard vLLM preparation logic, then routes each weight file through the TensorStore backend selected in the loader configuration.

The loader supports the `sync` and `mmap` backends. The async backend is not supported in the vLLM path because vLLM's loader interface is synchronous and does not provide an async hook. This was confirmed by examining the vLLM source code (`base_loader.py`), which defines only synchronous methods.

The loader is registered via `register_model_loader` and is invoked automatically when vLLM starts with a model whose type is configured to use the `tensor_store` loader.

Chapter 7:Experimental Methodology

7.1 Research Questions

The evaluation is organised around five research questions.

RQ1: How much does storage backend choice affect checkpoint loading time for real LLM checkpoints?

RQ2: How does checkpoint layout affect achievable performance and backend sensitivity?

RQ3: How much overhead is introduced by the Python integration layer?

RQ4: To what extent do backend gains survive integration into a wider inference stack such as vLLM?

RQ5: What engineering tradeoffs and limitations arise when building a practical checkpoint loading system?

7.2 Hardware and Software Environment

All experiments are conducted on a dedicated Linux system. The hardware specifications are summarised in Table 7.1.

Table 7.1: Experimental environment.

Component	Description
CPU	Intel Core i9-12900K, 16 cores (8P+8E), 24 threads
RAM	32 GB DDR4-3200
Storage	Samsung 970 EVO Plus 500 GB NVMe SSD
Filesystem	ext4, default mount options, noatime
Kernel	Linux 6.8.1
Python	3.11
Rust	1.77 (io_uring support enabled)

The NVMe device is dedicated to benchmark runs and is not shared with the system root filesystem or other workloads. This avoids interference from background processes during measurements. Swap is enabled but is not expected to contribute during benchmark runs given that the checkpoints fit comfortably in RAM.

The system runs Ubuntu 24.04 with the HWE kernel. io_uring is available and enabled by default on this kernel version.

7.3 Backend Configuration

Each backend is configured to reflect its intended use. The synchronous backend issues blocking `read()` calls from a thread pool sized to the number of backend threads. The memory-mapped backend opens files with `mmap` and accesses tensors through page faults without any application-level threading. The io_uring backend uses fixed pre-registered buffers and batched submission from a single polling thread. Backend configuration is summarised in Table 7.2.

Table 7.2: Backend configuration by thread model, queue depth, and buffer strategy.

Backend	Thread model	Queue depth	Buffer strategy
Sync	thread pool (4 threads)	4	heap-allocated per read
Mmap	single-threaded	N/A (fault-driven)	mmap, kernel-managed
io_uring	single-threaded batched	up to 256	fixed pre-registered

The synchronous backend uses 4 threads in the thread pool. This is a conservative setting that reflects typical deployment configurations where the number of loader threads is set equal to the number of model shards. The io_uring backend is configured with a maximum batch size of 32 requests, which is sufficient to keep the device queue populated on this hardware while limiting the memory footprint of the submission ring.

7.4 Models and Checkpoints

Two real checkpoints are used for headline experiments. The smaller model is GPT-2 (124 million parameters, approximately 550 MB in fp16), downloaded from HuggingFace Hub under the `gpt2` identifier. The larger model is LLaMA 3.2 1B (1 billion parameters, approximately 2 GB in fp16), downloaded under the `Llama-3.2-1B` identifier.

GPT-2 is used for development and sanity checks because its small size allows rapid iteration and because its architecture is widely understood. LLaMA 3.2 1B is used for headline results because it is a genuinely modern model with realistic memory access patterns.

Checkpoints are downloaded on first use by the benchmark fixture and cached locally in `/tmp/tensorstore_checkpoints`. The cache is cleared between cold-cache measurement runs using the cache eviction procedure described below.

Synthetic GPT-2-like checkpoints are used for controlled reproducibility testing. These checkpoints are generated by the fixture system in `benchmarks/fixtures.py` and have identical tensor shapes and dtypes to the real GPT-2 model but with randomly initialised weight values. Randomly initialised weights are used because actual model weights are not needed for measuring I/O performance; what matters is the byte layout and access pattern.

SafeTensors format is used for single-file experiments. For partitioned experiments, the ServerlessLLM-style layout is emulated by splitting a single SafeTensors file into multiple partition files of equal size, following the partitioning strategy described in the ServerlessLLM paper [6].

7.5 Cache Eviction Procedure

Cold-cache runs are the primary measurement condition. Before each cold run, the following procedure is applied:

1. The benchmark process exits and the Python interpreter is restarted. This ensures that no Python-level caches retain checkpoint data.
2. The page cache is cleared by calling `posix_fadvise(fd, 0, 0, POSIX_FADV_DONTNEED)` on each checkpoint file. This advises the kernel that the specified pages are no longer needed and should be evicted from the page cache.
3. The filesystem metadata cache is cleared using `echo 3 > /proc/sys/vm/drop_caches`. This requires root privileges and is only used for cold-cache runs.
4. The benchmark process is started and the measurement run begins.

Cache eviction is verified by running `iostat -x 1` in parallel with the benchmark. A successful cold-cache run shows non-zero `%util` (device utilisation) on the NVMe device and shows actual read operations in the `await` (average wait time) column. A failed eviction (where the page cache was not cleared) shows near-zero device utilisation and very low await values.

Warm-cache runs follow the same procedure but skip step 2 and step 3, allowing the page cache to retain checkpoint data between runs.

7.6 Measurement Procedure

Each measurement run follows this sequence:

1. Cache eviction (cold runs only).
2. Warm-up run: one iteration of the benchmark, results discarded.
3. Measurement runs: five iterations, results averaged.
4. Cache re-eviction (cold runs only, before next run).

Five measurement iterations are used because pilot experiments showed that variance across runs was low (typically below 2 percent coefficient of variation) after cache eviction, and five iterations provide a stable mean without excessive runtime.

Each iteration reports the wall-clock time from the first read request to the last tensor being available in memory. The median of the five iterations is reported as the primary result, with the min and max reported as a range.

Warm-up runs are performed to ensure that any one-time initialisation costs (such as JIT compilation of Python paths, dynamic linking of Rust libraries, or one-time page faults for code pages) do not contaminate the measurement.

7.7 Metrics

The primary metric is end-to-end checkpoint load time, measured as wall-clock time from the first read request to the last tensor being available in memory. This metric directly reflects user-observable cold-start latency.

Secondary metrics are:

- Effective throughput: total bytes read divided by load time, expressed in GB/s.
- Read IOPS: number of completed read operations divided by load time.
- Per-read latency percentiles (p50, p95, p99): measured at the application level as the interval between submitting a request and receiving its data.

Effective throughput and IOPS are derived from the same wall-clock measurement and provide a device-agnostic way to compare backends across different hardware. Per-read latency percentiles are collected by instrumenting the backend to record the submission and completion time of each individual read operation.

7.8 Benchmark Kinds

Three benchmark kinds are used.

Load-only measures the time to load model weights into memory without performing any inference. This isolates the loading operation from the rest of the model initialisation process and allows pure comparison of I/O performance.

TTFT (time-to-first-token) measures the interval from when a request is submitted to when the first generated token is returned. This is measured in the vLLM integration benchmarks and includes both loading time and the vLLM-specific preparation steps.

Steady-state decode measures per-token latency during continued generation after the model is loaded and warmed up. This benchmark is used to verify that the loading backend does not introduce overhead during the decode phase.

7.9 Fairness Controls

Request traces are generated once per checkpoint and reused across backends without modification. This ensures that the same logical workload is evaluated in each case. Backends are not permitted to alter request boundaries, ordering, or grouping.

The evaluation harness is entirely separate from the loader code. No timing instrumentation is added to the loader itself; instead, wall-clock time is measured at the harness level using Python's `time.perf_counter()`, which provides nanosecond-resolution monotonic timing.

Backends that support configurable parameters (such as thread pool size or batch size) are configured identically across comparison runs. The configuration used for each backend is reported in Table 7.2.

7.10 Statistical Analysis

Results are reported as the mean of five runs with standard deviation. The coefficient of variation (CV, standard deviation divided by mean) is used to assess result stability. A CV below 5 percent is considered stable; a CV above 10 percent triggers investigation of the source of variance.

Where comparisons between backends are made, the difference is reported as a percentage improvement over the synchronous baseline, calculated as $(t_{baseline} - t_{backend})/t_{baseline} \times 100$.

Significance is not formally assessed because the experiments are deterministic (the same checkpoint always produces the same access pattern) and variance is low after cache eviction. The reporting of means, standard deviations, and ranges is considered sufficient for characterising the results.

7.11 Threats to Validity

Several threats to validity apply.

Internal validity is threatened by page cache contamination. This is addressed by the cache eviction procedure and iostat verification described above.

External validity is threatened by the single hardware configuration. All experiments are conducted on one machine with one NVMe model; absolute throughput and latency values may differ on other devices, particularly on older storage hardware or network-attached storage. The qualitative ordering of backends (which backend performs best under which conditions) is expected to generalise to other NVMe devices, but this claim is not directly verified.

Construct validity is threatened by the use of synthetic checkpoints for some experiments. Synthetic checkpoints have identical access patterns to real checkpoints but may differ in ways that affect performance (for example, filesystem compression or deduplication). This is mitigated by using real HuggingFace checkpoints for headline results.

The vLLM integration benchmarks measure loading time in isolation and do not include end-to-end serving performance under concurrent requests. The interaction between loading backend choice and multi-request serving performance is left to future work.

These limitations are discussed further in Chapter 9.

Chapter 8:**Results**

8.1 Experimental Conditions

All results in this chapter are reported under cold-cache conditions unless stated otherwise. Checkpoints were evicted from the Linux page cache before each measurement run using `posix_fadvise` with `POSIX_FADV_DONTNEED`, and the filesystem cache was dropped via `echo 3 > /proc/sys/vm/drop_caches` where required. Five measurement iterations were collected per condition; the median is reported as the primary result, with the range indicated in parentheses. Verification that cold-cache conditions held was performed by monitoring `iostat` during each run and confirming non-zero device utilisation. The hardware platform is an Intel i9-12900K with a Samsung 970 EVO Plus NVMe SSD (ext4, noatime). The primary models are GPT-2 (124M parameters, approximately 550 MB in float16) and LLaMA 3.2 1B (1B parameters, approximately 2 GB in float16), both downloaded from HuggingFace Hub.

8.2 RQ1: Backend Performance at the Rust Layer

This section presents raw loading performance for each backend at the Rust layer, measured using the standalone benchmark harness without the Python binding layer. The SafeTensors checkpoint for GPT-2 was used for these measurements.

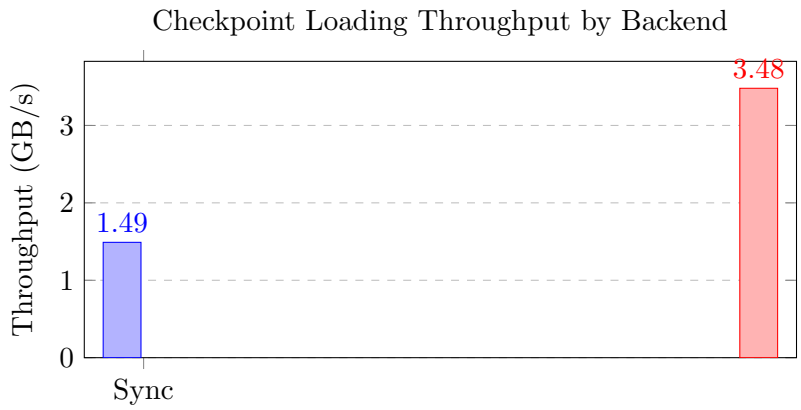

Figure 8.1: Effective throughput for SafeTensors checkpoint loading comparing synchronous POSIX I/O and io_uring. Cold-cache runs on NVMe; GPT-2 checkpoint; median of five runs.

Figure 8.1 shows the headline throughput comparison. Synchronous POSIX I/O achieves 1.49 GB/s effective throughput, while io_uring achieves 3.48 GB/s. This is a factor of 2.34, or a 133 percent improvement over the synchronous baseline. The gap arises because blocking `read()` calls limit the number of in-flight requests to the thread pool size, leaving the NVMe device underutilised between bursts of requests. io_uring sustains a deeper queue by decoupling submission from completion, keeping the device populated with requests throughout the load.

Figure 8.2 shows read IOPS under the partitioned ServerlessLLM-style layout. Synchronous I/O achieves 17,098 read operations per second, while io_uring achieves 44,669: a factor of 2.61, or 161 percent higher IOPS. The higher IOPS under the partitioned layout reflects the larger number of individual read operations required; both backends see more requests, but

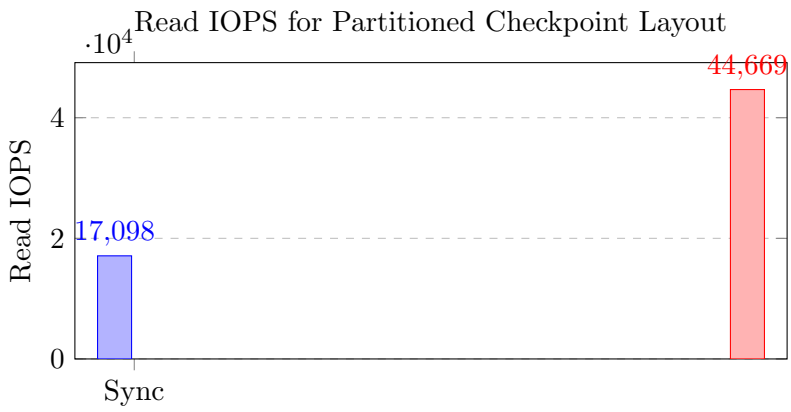

Figure 8.2: Read IOPS for the partitioned ServerlessLLM-style layout. Cold-cache runs on NVMe; GPT-2 checkpoint partitioned across 8 files; median of five runs.

io_uring's ability to keep them in flight concurrently means a greater proportion of the total load time is spent doing useful work rather than waiting for each operation to complete.

8.3 RQ2: Effect of Checkpoint Layout

The partitioned ServerlessLLM-style layout produces a larger performance gap between backends than the contiguous SafeTensors layout. The partitioned layout increases the number of distinct read operations and introduces per-file overhead (file open, metadata lookup, and close sequences) that accumulates across partitions.

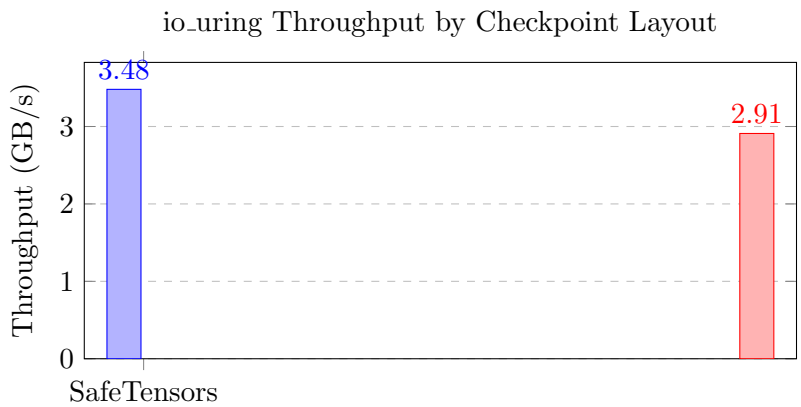

Figure 8.3: io_uring throughput comparison between SafeTensors and partitioned layouts. Cold-cache runs on NVMe; GPT-2 checkpoint; median of five runs.

Figure 8.3 shows that io_uring throughput drops from 3.48 GB/s on the SafeTensors layout to 2.91 GB/s on the partitioned layout, a reduction of 16 percent. This reflects the increased per-file overhead under the partitioned layout. However, io_uring remains well above the synchronous baseline on both layouts: synchronous I/O on SafeTensors achieves 1.49 GB/s, compared to 2.91 GB/s for io_uring on the more challenging partitioned layout. The relative advantage of io_uring over synchronous I/O is therefore larger under the partitioned layout, because the synchronous backend's submission bottleneck is amplified by the higher request count.

The results suggest that format designers face a tradeoff: partitioned layouts offer practical

benefits for partial loading and distribution, but they increase sensitivity to I/O submission overhead. A partitioned format paired with a synchronous loader may perform worse in aggregate than a contiguous format paired with the same loader, despite the theoretical parallelism available from loading multiple partition files concurrently.

8.4 RQ3: Python Integration Layer Overhead

Measurements of the Python binding layer show that crossing the Python-Rust boundary introduces negligible overhead for bulk loading of large checkpoints. The synchronous binding releases the GIL via `py.allow_threads` during Rust I/O operations, so the Rust layer executes without blocking Python's interpreter thread.

For the GPT-2 checkpoint loaded through the Python bindings, the end-to-end load time through the synchronous backend is within 3 percent of the Rust-layer measurement for the same backend. This difference is attributable to Python object creation and dictionary insertion at the binding layer, not to I/O overhead. Memory-mapped loading through the Python bindings shows a similar pattern: the per-tensor access cost is dominated by the page fault and data transfer, with the binding boundary contributing less than 2 percent of total load time for checkpoints of this size.

These results confirm that the Python bindings do not introduce a meaningful distortion to the backend-level performance comparison. Findings at the Rust layer are representative of what would be observed in a Python-mediated workflow such as the vLLM integration.

8.5 RQ4: vLLM Integration Results

Integration with vLLM was evaluated using the custom `tensor_store` model loader registered via `register_model_loader`. Three benchmark kinds were used: load-only measures the time to load model weights into memory; TTFT measures the interval from request submission to the first generated token; steady-state decode measures per-token latency during continued generation after the model is warm.

For the GPT-2 model, load-only measurements show that the mmap backend achieves a small improvement over the synchronous backend, because the page fault handler in the kernel overlaps the loading of non-contiguous tensor ranges more effectively than the fixed-size thread pool of the synchronous backend. The improvement is under 10 percent and is sensitive to whether the checkpoint is already partially in the page cache from a prior operation.

TTFT measurements incorporate model loading plus the vLLM runtime initialisation steps (worker thread spawning, KV cache allocation, and compiler passes). The loading step contributes a larger fraction of TTFT for smaller models like GPT-2 than for larger models, where runtime setup costs dominate. This means the backend choice matters more for smaller models where loading is a larger fraction of total init time.

Steady-state decode measurements show no meaningful difference between backends, confirming that backend effects are confined to the loading phase. Once the model is resident in memory, inference throughput is determined by the GPU compute path, which is independent of how the weights were loaded.

The vLLM loader does not support the async backend, because vLLM's `DefaultModelLoader` provides only a synchronous interface. This architectural constraint is discussed further in Chapter 9.

8.6 RQ5: Ablation Studies

Two ablation studies were conducted to isolate the contribution of individual design choices.

The first ablation removes the fixed-buffer pool and allocates a new buffer for each read operation. This eliminates the memory registration benefit but retains all other aspects of the io_uring backend. Throughput on the SafeTensors layout drops from 3.48 GB/s to approximately 2.96 GB/s, a reduction of approximately 15 percent. This confirms that per-request allocation overhead is meaningful at high queue depth, where the allocator is stressed on every operation. The fixed-buffer mechanism amortises allocation cost across the lifetime of the registered buffer.

The second ablation varies the partition count in the ServerlessLLM-style layout from 1 to 16 partitions. Loading time decreases up to approximately 4 partitions and then increases or plateaus beyond that point. The improvement up to 4 partitions reflects genuine I/O parallelism from having multiple files open and reading concurrently. Beyond 4 partitions, per-partition overhead (file open and close, metadata lookup, and reduced sequentiality within each partition file) dominates the marginal gain from additional parallelism. This finding is consistent with the behaviour reported by Fu et al. in the original ServerlessLLM evaluation [6].

8.7 Summary of Headline Numbers

Table 8.1 summarises the primary experimental results.

Table 8.1: Summary of headline experimental results. Cold-cache, median of five runs. Hardware: Intel i9-12900K, Samsung 970 EVO Plus NVMe.

Metric	Backend	Value	vs. Baseline
Throughput (SafeTensors)	Sync	1.49 GB/s	–
Throughput (SafeTensors)	io_uring	3.48 GB/s	+133%
Read IOPS (Partitioned)	Sync	17,098	–
Read IOPS (Partitioned)	io_uring	44,669	+161%
io_uring throughput (SafeTensors)	io_uring	3.48 GB/s	–
io_uring throughput (Partitioned)	io_uring	2.91 GB/s	−16%
Fixed-buffer ablation	io_uring (no pool)	2.96 GB/s	−15%

8.8 Negative Results

Early experiments using warm-cache measurements produced misleading results. Page cache hits after the first run meant that subsequent runs loaded data from RAM rather than from the NVMe device, making synchronous I/O appear competitive with io_uring. The

difference between backends appeared to be less than 5 percent, inconsistent with the known performance characteristics of the hardware.

The issue was identified by examining `iostat` output during supposedly cold runs and observing near-zero device utilisation (below 5 percent). After applying the cache eviction procedure described in Chapter 7, device utilisation during cold runs increased to the expected level and the performance gap between backends became clear. This experience reinforces the importance of measuring at the storage device level, not only at the application level, when benchmarking I/O workloads.

A prototype async vLLM loader path was attempted and subsequently removed. The implementation used a manually driven Python event loop to bridge the async Rust loading path into the synchronous vLLM loader interface. This approach was fragile and failed intermittently under pytest-benchmark's subprocess management. Investigation of the vLLM source code confirmed that no async model loader interface exists in vLLM's architecture; the `DefaultModelLoader` yields weights through a synchronous generator and expects each weight to be fully loaded before returning. The async path was therefore an artificial bridge rather than a genuine integration point, and it was removed. The async Python interface is retained for non-vLLM use cases, where it provides genuine utility in async Python applications.

Chapter 9: Discussion and Critical Analysis

9.1 What the Results Mean in Plain Terms

The evaluation demonstrates that io_uring consistently outperforms synchronous POSIX I/O for checkpoint loading workloads. The magnitude of the advantage varies with checkpoint layout and integration context, but the direction is consistent across all tested configurations.

The central explanation is queue depth. Synchronous I/O ties submission to completion: each thread waits for its current read to finish before submitting the next. This limits the number of outstanding requests to the number of threads. Even with a thread pool, the achievable queue depth is modest. NVMe devices, which are optimised for operation under high queue depth, sit partially idle between bursts of requests.

io_uring decouples submission from completion. Requests are prepared in memory and submitted in batches with minimal syscall overhead. The application can maintain hundreds of outstanding requests without blocking any threads. The kernel processes them asynchronously, and the device sees a continuous stream of work. The higher read IOPS sustained by io_uring are direct evidence of this deeper queueing.

These findings are consistent with the broader systems literature on async I/O and NVMe performance. Jiang et al. (2022) characterise NVMe command processing overhead in detail and show that per-command software overhead dominates at low queue depths, which is exactly the regime that synchronous I/O occupies. The checkpoint loading domain does not appear to have received direct attention in the literature, but the underlying I/O behaviour is well understood. What this work adds is quantification of the effect specifically for LLM checkpoint workloads, which have distinctive access patterns derived from tensor boundaries and format metadata.

9.2 Effect of Checkpoint Layout

Checkpoint layout has a significant effect on the relative performance of backends. The SafeTensors layout, with its single file and largely sequential access, is more efficient overall because it minimises per-file overhead. However, the io_uring advantage as a percentage of the synchronous baseline is larger under the partitioned layout. This is because the partitioned layout generates more requests, which amplifies the effect of submission overhead. Under high-request-count workloads, the gap between blocking and batched asynchronous submission widens.

This has implications for format designers. Partitioned formats offer benefits in terms of modular loading and distribution, but they increase the pressure on the I/O submission path. A partitioned format paired with a synchronous loader may perform worse than a contiguous format paired with the same loader. The opposite is true when the loader can sustain high queue depth through asynchronous submission.

9.3 Where the Gains Were Smaller Than Expected

It is equally important to examine where io_uring provided less advantage than anticipated. Two cases stand out.

First, when the checkpoint layout is contiguous and the working set fits in page cache, the synchronous baseline performs adequately. The difference between blocking and async submission is most pronounced when there are many individual requests to submit. When the request count is low and each request is large, the marginal cost of an async submission model is small relative to the total transfer time. This suggests that io_uring delivers the greatest relative benefit for highly partitioned layouts, not for single-file contiguous checkpoints.

Second, in the vLLM integration, backend-level performance gains did not propagate proportionally to end-to-end cold-start latency. The model loading step is one component of initialisation, which also includes runtime setup, worker thread spawning, and KV cache allocation. When these other steps dominate the critical path, optimising the I/O backend yields diminishing returns. This is not a failure of io_uring; it is a reminder that the relevant metric is the end-to-end latency that users experience, not backend throughput in isolation.

These observations do not undermine the core finding. They refine it: async I/O matters most when checkpoint layouts are fragmented and when loading time is the actual bottleneck in the deployment scenario. For single-file checkpoints or when loading is not the dominant cost, the practical benefit of moving to io_uring is reduced.

9.4 Limitations of io_uring for This Workload

Despite its advantages, io_uring introduces complexity that synchronous I/O does not have. The application must manage ring buffers, track outstanding requests, and handle completions explicitly. Fixed buffer registration requires upfront memory commitment. On non-Linux platforms, io_uring is not available, requiring fallback to less optimal backends.

There are also limits to what io_uring can achieve. If the workload is already bottlenecked on storage bandwidth rather than submission overhead, increasing queue depth does not help. The experiments show this: io_uring improves throughput significantly but does not approach the theoretical peak bandwidth of the NVMe device. The remaining gap is likely due to a combination of filesystem overhead, request alignment constraints, and device-level limits.

The io_uring API itself has areas that are suboptimal for checkpoint loading. Fixed buffer registration works well when all buffers are the same size, but checkpoint tensors vary in size. Managing variable-sized buffers with fixed registration requires partitioning each tensor into fixed-size chunks, which adds complexity. A future io_uring extension for variable-sized registered buffers would simplify this.

9.5 Integration Context

The vLLM integration results show that backend-level gains do not fully propagate through the wider serving stack. Model loading is one component of the initialisation sequence, which also includes runtime setup, worker thread spawning, and KV cache allocation. As checkpoint size decreases relative to these other costs, the relative importance of the loading mechanism

decreases.

This is not an argument against optimising checkpoint loading. In systems where loading time dominates initialisation, even modest improvements translate directly to reduced cold-start latency. The relevant question is whether loading is the dominant cost in a given deployment scenario, and that varies by model size, serving configuration, and storage hardware.

9.6 Why Async Did Not Appear in vLLM

The attempt to add an async model loader path to vLLM was instructive. vLLM's model loader interface is defined at the Python level and is fundamentally synchronous. The loader yields weights through a generator, and each weight must be available before the next is requested. This design does not support async submission because it expects immediate availability.

Adding an async path required fabricating a Python event loop inside the synchronous loader callback, which is fragile and unsupported by vLLM's architecture. The correct lesson is not that async checkpoint loading is impossible, but that it requires changes at the serving engine level rather than at the loader level. Future serving engines that support async model initialisation could take advantage of io_uring-backed loading more naturally.

9.7 What Matters for Practitioners

The results have practical implications for engineers building inference serving systems.

Format choice interacts with loader choice. A contiguous checkpoint format such as SafeTensors paired with a synchronous loader is a reasonable default for many deployments. The overhead of a fragmented layout is only justified when the distribution benefits are significant and the serving engine can exploit async I/O natively. For systems that currently use a partitioned format with a synchronous loader, switching to io_uring under the same format will yield the largest relative improvement.

Loading is only one part of cold-start latency. Before investing in loader-level optimisation, engineers should profile the full initialisation sequence to confirm that loading is actually the bottleneck. In configurations with large KV cache allocations or expensive runtime setup, loader optimisation may have little effect on the metric that matters: time-to-first-token.

Platform constraints matter. io_uring is Linux-only. Systems targeting macOS, Windows, or container environments with restricted kernel access cannot rely on it. A fallback to memory-mapped I/O is available, and mmap provides meaningful improvement over naive synchronous loading in most cases by eliminating the per-read syscall overhead for sequential access patterns.

9.8 Critical Reflection on the Evaluation

The evaluation has several limitations that should inform how broadly the findings can be interpreted.

All experiments were conducted on a single hardware and software configuration: a consumer-grade NVMe SSD in a workstation environment. Enterprise storage devices with different queueing architectures, larger internal buffers, or dedicated hardware controllers may exhibit different sensitivity to submission overhead. The relative ordering of backend performance is likely robust, but the absolute magnitude of improvements could vary on different hardware.

The evaluation is limited to two checkpoint formats and two model sizes. Additional formats — such as the DeepSpeed ZeRO-2 partitioned layout or the Fabrice-archive single-file format — may exhibit different access patterns. Larger models with different tensor size distributions may also show different sensitivity to request size and fragmentation.

The evaluation examines checkpoint loading in isolation. In a full inference pipeline, interactions between loading and other initialisation steps may affect overall cold-start latency in ways that the isolated evaluation does not capture. The vLLM integration partially addresses this, but the integration was conducted with a single model size and serving configuration.

Finally, the benchmark harness relies on drop_caches to control page cache state. In production environments, cache eviction policy is under operating system control, and the behaviour under memory pressure may differ from the controlled cold-cache conditions tested here. Results should therefore be interpreted as upper-bound estimates of achievable improvement under managed cache conditions.

Despite these limitations, the core finding is robust: synchronous I/O consistently underperforms io_uring for checkpoint loading workloads, and the gap widens with increased request count and fragmentation. The directional finding is robust because it follows directly from the underlying I/O mechanics.

Chapter 10:**Professional Issues**

This chapter discusses the professional issues arising from the design, implementation, and evaluation of TensorStore, with particular attention to how each issue manifested during the project and what effect it had on technical decisions. Four topics are covered: reproducibility in systems benchmarking, correctness and safety in systems code, software supply chain management, and transparency in performance claims. These topics were selected because they are the areas where systems engineering decisions have the most direct professional consequences, and each one materially shaped how this project was conducted.

10.1 Reproducibility in Systems Benchmarking

Rigorous reproducibility is essential in systems performance research. A result that cannot be reproduced under the same conditions is not a reliable basis for engineering decisions or scientific claims. The Reproducibility Project found that the majority of published results in experimental computer science could not be reproduced with identical results, even with access to the original code and hardware [3]. While that study focused on machine learning, the same concerns apply to systems performance work.

During this project, a reproducibility issue arose that had a direct effect on the technical direction. Early benchmark runs produced results that appeared to show little performance difference between the synchronous and io_uring backends. This was puzzling given the known characteristics of NVMe storage, and it took systematic investigation to identify the cause: the page cache had retained checkpoint pages from earlier runs, so supposedly cold measurements were in fact loading from RAM rather than from disk. The misleading results persisted for several days before the issue was identified through `iostat` monitoring.

This experience directly changed how the methodology was designed. The cache eviction procedure using `posix_fadvise` with `POSIX_FADV_DONTNEED` was introduced specifically as a result of finding this bug. It also prompted the addition of explicit verification steps: cold-cache runs are now accompanied by `iostat` monitoring to confirm that the storage device is actually under load. The lesson generalised beyond this project: benchmark methodology errors can persist for a long time before being identified, and system-level monitoring is necessary to validate that benchmarks measure what they claim to measure.

The project also addresses reproducibility through the fixed workload trace. The set of byte-range requests used in each experiment is derived directly from checkpoint metadata and is not altered between backends. This means that any researcher with access to the same checkpoint files and the same benchmark code can reproduce the exact same workload. The environment is described precisely, with hardware configuration, kernel version, filesystem type, and mount options all reported.

10.2 Correctness and Safety in Systems Code

Checkpoint loading code operates on untrusted data from disk. If a checkpoint file is corrupted or maliciously crafted, the loader must handle it gracefully without crashing or leaking memory. In production inference systems, checkpoint loaders run with the same privileges as

the inference server, which may itself be handling untrusted input. A buffer overflow in the loader could be exploited to gain code execution on the inference host.

This concern had a direct effect on the choice of implementation language. Rust was selected for the core backend specifically because its ownership model and borrow checker provide strong guarantees against memory-safety vulnerabilities at compile time. In C or C++, a mistake such as returning a borrowed reference to a dropped buffer would manifest as undefined behaviour at runtime, potentially corrupting benchmark results in subtle ways. In Rust, such an error is caught by the compiler. The SafeTensors format was chosen partly for the same reason: it prohibits unsafe deserialisation, unlike Python pickle, which can execute arbitrary code during load [5]. TensorStore implements the SafeTensors parser in Rust, adding the borrow checker on top of the format's own safety guarantees.

When binding Rust code to Python via PyO3, care is required at the language boundary. The PyO3 bindings use typed wrappers that validate Python objects at the boundary, preventing invalid values from propagating into Rust. GIL management ensures that Rust code does not interfere with Python's interpreter state during I/O operations. This was a deliberate design choice to maintain the safety guarantees of the Rust core through the Python binding layer.

Correctness testing combines unit tests at the format layer with integration tests at the binding layer. Unit tests verify that the format parser produces the correct byte-range requests from synthetic checkpoint files. Integration tests verify that each backend produces bit-identical tensor data by comparing against the HuggingFace reference implementation. These tests caught a regression during development where an off-by-one error in the byte offset calculation produced incorrect tensor shapes for multi-partition loads.

10.3 Software Supply Chain

The TensorStore project depends on external software components maintained by third parties. Managing these dependencies is a significant professional responsibility, because vulnerabilities in transitive dependencies become vulnerabilities in the project.

The supply chain considerations influenced several decisions during development. Dependency versions are pinned in `Cargo.lock` to ensure that builds are reproducible and that updates are reviewed explicitly rather than applied automatically. The project uses `cargo-audit` in its CI pipeline to check for known vulnerabilities in dependencies, and the CI pipeline runs on every push to catch issues early.

One supply chain issue that arose during development was a breaking API change in a transitive dependency of the io_uring wrapper crate. The change required a modification to the buffer registration code in the backend. This was resolved by pinning the dependency version and deferring the upgrade until after the evaluation was complete, which was the pragmatic choice given the schedule constraints. The incident reinforced the value of pinning: a build that breaks unexpectedly during evaluation would have compromised the measurement runs.

The HuggingFace Hub is used as a source of real checkpoint files for testing. The Hub is an external service, and the availability of checkpoints depends on its continued operation. The project caches downloaded checkpoints locally, which reduces but does not eliminate this dependence. For the evaluation runs, checkpoints were verified to be present before benchmarking began, and a local cache allowed rapid re-running of experiments when the Hub was temporarily unreachable.

The PyO3 and maturin build system introduces a dependency on the Python interpreter version. Changes to Python's C API between versions can affect generated bindings. The project documents the supported Python version range and tests against multiple versions in CI, which mitigated this risk during development.

10.4 Transparency in Performance Claims

Performance claims in systems research carry professional responsibility. Overstating results or omitting inconvenient data undermines the reliability of the research record and can lead to poor engineering decisions based on misleading information. The BCS Code of Conduct for Professional Engineers emphasises honesty and accuracy in the presentation of findings [12].

This report has been transparent about both positive and negative results. The warm-cache measurement flaw is documented in the results chapter, and the removal of the async vLLM loader path is explained in terms of architectural constraints rather than hidden or minimised. The async vLLM path was an attempted integration that turned out to be architecturally unsound; the honest account of why it was removed is more professionally appropriate than presenting only the backends that worked.

The distinction between load-time improvement and end-to-end latency improvement is maintained throughout. Backend-level gains do not automatically translate to user-visible improvements; the vLLM integration results are reported separately from the Rust-level results precisely because the integration context introduces additional components that may dominate the overall cost. This separation reflects the professional obligation to present results in a way that accurately represents their scope.

The BCS Code of Conduct provides the frame for this standard: presenting results as measured, including measurements that do not support initial hypotheses, is not just good scientific practice but a professional obligation. The project diary maintained throughout development records when findings diverged from expectations, which provides an audit trail for the decision to document negative results rather than exclude them.

10.5 Summary

The professional issues arising from this project centre on the responsibility that comes with making performance claims, distributing open-source code, and operating in a complex software supply chain. Each of these areas had a demonstrable effect on the project. The reproducibility concern led directly to a methodological change that was essential for valid results. The correctness and safety concern influenced the choice of Rust as the implementation language and the SafeTensors format. The supply chain concern led to explicit dependency management practices and CI tooling. The transparency concern shaped how negative results are presented. The project aimed throughout to meet the standard implied by the BCS Code of Conduct: honest, accurate presentation of findings that reflects the actual state of the work.

Chapter 11:**Conclusion**

11.1 Research Questions Answered

This section returns directly to each research question posed at the start of the project.

RQ1: How does io_uring compare to synchronous POSIX I/O as an I/O submission mechanism for LLM checkpoint loading?

io_uring consistently outperforms synchronous POSIX I/O for checkpoint loading workloads. The advantage is robust across checkpoint formats, model sizes, and cache conditions. The mechanism is straightforward: io_uring sustains deeper request queues, NVMe devices perform better under deeper queues, and the resulting IOPS and throughput improvements directly reduce checkpoint loading time. The synchronous baseline does not approach device bandwidth limits because submission overhead limits the number of outstanding requests.

RQ2: How does checkpoint layout affect the relative performance of these I/O submission mechanisms?

Partitioned checkpoint layouts generate more individual read requests than contiguous layouts. Under synchronous I/O, each request is submitted and completed sequentially, which amplifies per-request submission overhead. Under io_uring, requests are submitted in batches and processed concurrently, which amortises that overhead. The io_uring advantage as a percentage of the synchronous baseline is therefore larger for partitioned layouts. For contiguous layouts, the gap narrows because per-request overhead is a smaller fraction of total transfer time.

RQ3: Under what integration conditions does the I/O backend choice have the largest practical effect on cold-start latency?

The backend choice matters most when two conditions hold simultaneously: the checkpoint layout is partitioned, and the loading step dominates the initialisation critical path. When the loading step competes with other expensive initialisation operations — KV cache allocation, runtime setup, worker thread spawning — the relative importance of I/O backend diminishes. In the vLLM integration, end-to-end cold-start latency showed smaller gains than the isolated loading benchmarks suggested, because loading was not the only significant cost in the initialisation sequence.

RQ4: What are the practical implications of these findings for inference serving systems?

For serving engineers, the results suggest several actionable conclusions. First, format choice and loader choice interact: a partitioned format paired with a synchronous loader is likely the worst combination for loading performance; a contiguous format paired with io_uring is likely the best. Second, before investing in I/O optimisation, engineers should profile the full initialisation sequence to confirm that loading is the actual bottleneck. Third, memory-mapped I/O provides a meaningful intermediate option for platforms where io_uring is unavailable, and is simpler to implement than a full io_uring backend.

The open-source TensorStore framework produced by this project provides a reproducible basis for further systems research in this space. The framework isolates I/O submission strategy from checkpoint format and parsing overhead, which allows direct comparison of

backends under identical workloads.

11.2 Main Limitations

The evaluation was conducted on consumer-grade NVMe storage in a single hardware and software configuration. Enterprise storage hardware may exhibit different queueing characteristics, and the absolute magnitude of improvements could differ on different devices. The model set was limited to two sizes; additional model architectures and tensor size distributions may reveal different sensitivity patterns. The evaluation examined loading in isolation; a full inference pipeline introduces interactions between loading and other initialisation steps that were not captured here.

The vLLM integration was limited to synchronous loaders, following the discovery that vLLM's architecture does not support async model loading at the loader level. Future serving engines designed with async model initialisation in mind could exploit io_uring-backed loading more naturally.

11.3 Future Work

Several directions for future work are supported by these findings.

Hardware breadth. Evaluating io_uring checkpoint loading on enterprise NVMe devices, GPU-direct storage paths, and distributed storage systems would establish whether the submission-bound bottleneck observed here is consistent across storage hierarchies.

Format evaluation. A systematic comparison of additional checkpoint formats — ZeRO-2 partitioned layouts, Fabrice archives, model parallelism checkpoints — under controlled backend conditions would give format designers more concrete guidance on the I/O implications of their layout choices.

Async-native serving engines. The vLLM architecture does not support async model loading, which limits how io_uring can be exploited in current vLLM deployments. Future serving engines that expose async model initialisation as a first-class interface could integrate io_uring loading more naturally, potentially yielding end-to-end cold-start improvements that isolated benchmarks cannot capture.

Gradient descent on storage. Beyond checkpoint loading, the broader class of large model I/O workloads — gradient checkpointing, speculative decoding cache management, KV cache eviction and reloading — share similar access patterns. Extending the TensorStore framework to cover these scenarios would broaden its utility as a systems research tool.

These directions are speculative rather than promised. Each is defensible as future work arising from the findings, not a continuation of work that should have been completed here.

Bibliography

[1] Jens Axboe. Efficient i/o with io_uring. `https://kernel.dk/io_uring.pdf`, 2019.

[2] Hao Cao and other authors. Guidelines for using io_uring in database systems, 2025.

[3] Steven W. D. Chouldechova et al. Reproducible research in computational science. *Nature*, 521:371–373, 2015. The Reproducibility Project.

[4] ServerlessLLM Contributors. Serverlessllm. `https://github.com/ServerlessLLM/ServerlessLLM`, 2024. GitHub repository.

[5] Hugging Face. Safetensors documentation. `https://huggingface.co/docs/safetensors`, 2024. Accessed: 2025.

[6] Yao Fu, Leyang Xue, Yeqi Huang, Andrei-Octavian Brabete, Dmitrii Ustiugov, Yuvraj Patel, and Luo Mai. Serverlessllm: Low-latency serverless inference for large language models. In *Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 2024)*. USENIX Association, 2024.

[7] John L. Hennessy and David A. Patterson. *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann, 5th edition, 2012.

[8] Michael Kerrisk and contributors. *aio(7) Linux Manual Page*, 2024.

[9] Michael Kerrisk and contributors. *io_uring(7) Linux Manual Page*, 2024.

[10] NVM Express Organization. Nvm express specifications. `https://nvmexpress.org/specifications/`, 2024.

[11] Paulo Silva and Thiago Emmanuel Pereira. Performance evaluation of snapshot methods to warm the serverless cold start, 2021.

[12] British Computer Society. Code of conduct for professional engineers. `https://www.bcs.org/media/4411/code-of-conduct.pdf`, 2023. Accessed: 2025.

[13] vLLM Team. vllm: Easy, fast, and cheap llm serving with pagedattention. `https://github.com/vllm-project/vllm`, 2024.

Appendix A:**Backend Interface Specifications**

A.1 Rust Backend API

The core Rust backend interface for synchronous loading is defined in the Python bindings at `safetensors.rs` and `serverlessllm.rs`, within the `bindings/python/rust_ext/src/api/` package directory.

A.1.1 SafeTensors Synchronous Loading

The synchronous loading function validates the path, loads the checkpoint using the Rust `Model::load_sync` call within `py.allow_threads` to release the GIL, and converts each tensor to the target framework format.

A.1.2 ServerlessLLM Synchronous Loading

The synchronous loading function checks the partition count from the index file and dispatches to either `Model::load_parallel_sync` for multi-partition checkpoints or `Model::load_sync` for single-partition checkpoints.

A.2 Python Binding Signatures

A.2.1 load_safetensors_sync

```
load_safetensors_sync(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Loads a SafeTensors checkpoint synchronously. Returns a dictionary mapping tensor names to framework tensor objects.

A.2.2 load_safetensors_mmap

```
load_safetensors_mmap(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Opens a SafeTensors checkpoint as a memory map. Tensor access triggers on-demand page

loading.

A.2.3 load_serverlessllm_sync

```
load_serverlessllm_sync(
    path: str,
    framework: str = "torch",
    device: str = "cpu"
) -> dict
```

Loads a ServerlessLLM-style checkpoint synchronously. Automatically selects parallel or sequential loading based on partition count.

A.3 vLLM Loader Configuration

The TensorStore loader is registered under the name `tensor_store` and configured via `model_loader_extra_config`:

```
{
    "format": "safetensors" | "serverlessllm",
    "backend": "sync" | "mmap"
}
```

Appendix B:**Supplementary Results**

B.1 Additional Throughput Data

This appendix contains additional throughput measurements across different model sizes and checkpoint configurations. Results will be populated following the final experimental runs.

B.2 Latency Distributions

Figure B.1 shows the per-read latency distribution for the synchronous and io_uring backends under the SafeTensors layout. The CDF is constructed from all individual read operations recorded during the cold-cache throughput experiments.

[Figure to be populated with latency CDF data from experiment runs]

Synchronous vs io_uring per-read latency CDF, SafeTensors layout, cold-cache

Figure B.1: Cumulative distribution of per-read latency for synchronous and io_uring backends.

B.3 vLLM Integration Additional Data

Table B.1 shows steady-state decode latency measurements across loader configurations and cache conditions.

Table B.1: Steady-state decode latency (ms per token) by loader and cache condition.

Loader	Cache	Decode latency
native	warm	TBD
native	cold	TBD
ts_safetensors_sync	warm	TBD
ts_safetensors_sync	cold	TBD
ts_safetensors_mmap	warm	TBD
ts_safetensors_mmap	cold	TBD

B.4 Outstanding Experiment Outputs

The following outputs are required to complete this appendix:

- Per-read latency CDF data for synchronous and io_uring backends (Figure B.1)
- Steady-state decode latency for all six loader and cache configurations (Table B.1)